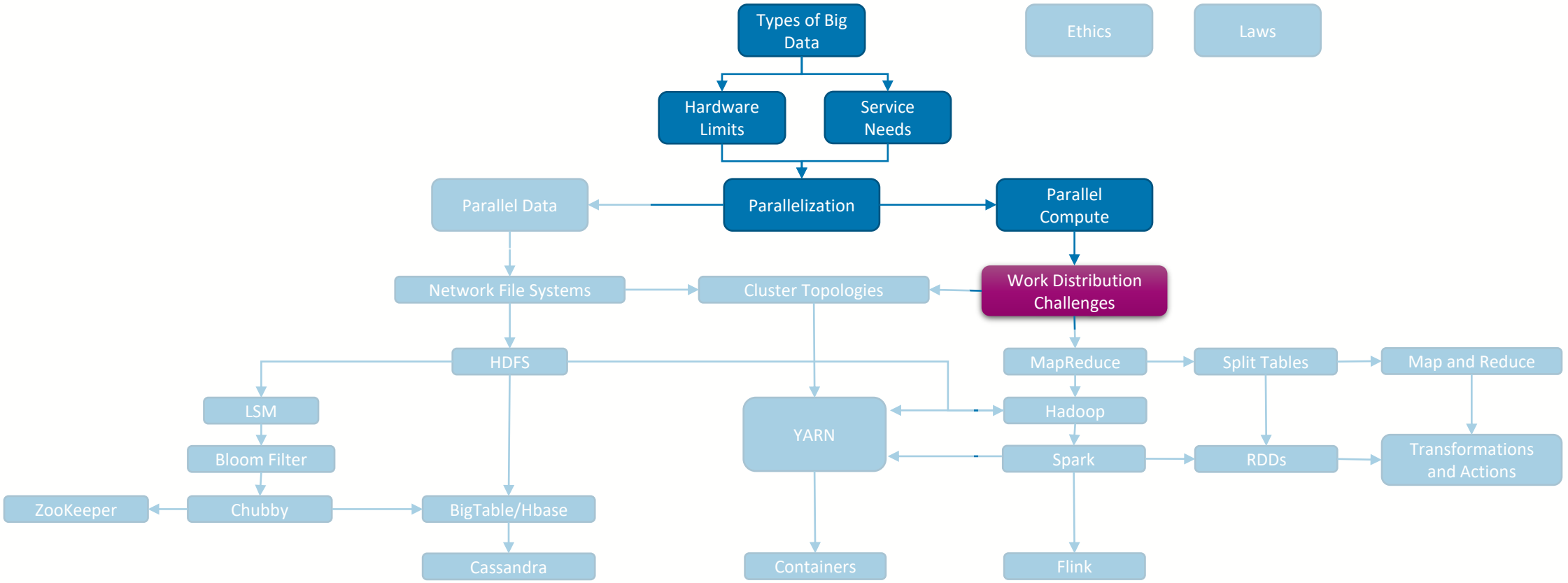
W2-01: Work Distribution Challenges

COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreadie

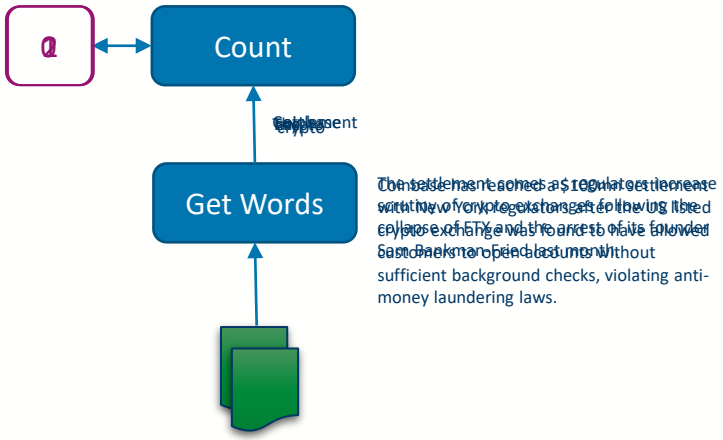
University of Glasgow

Where are we?



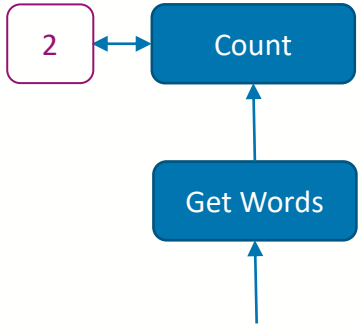
Problems with Parallelism

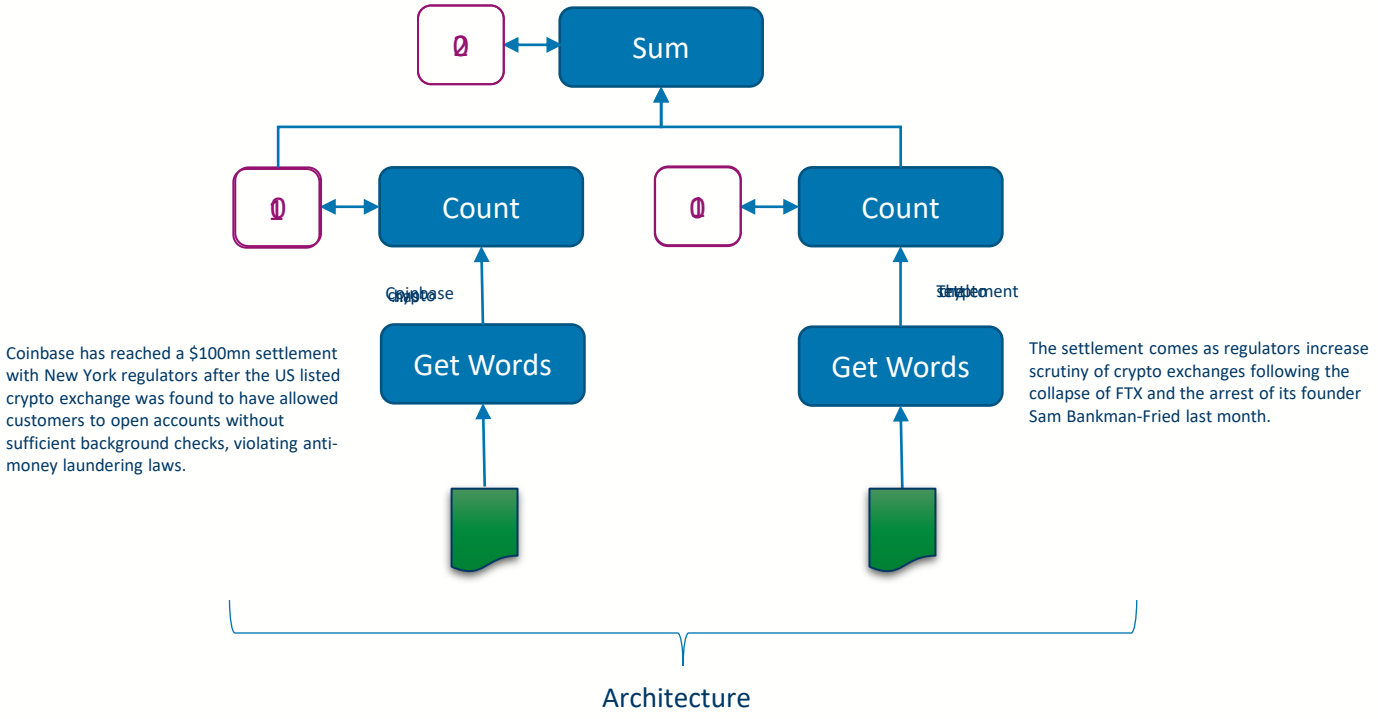
- Lets assume that we have a set of 2 text documents and we want to count the unique number instances of a single word, e.g. ‘crypto’
  - The Serial solution would involve creating a counter to track the number of the word ‘crypto’ that has been seen, then for each word we check if it is ‘crypto’, and if so increment the counter



Problems with Parallelism

- Lets assume that we have a set of 2 text documents and we want to count the unique number instances of a single word, e.g. ‘crypto’
  - The Serial solution would involve creating a counter to track the number of the word ‘crypto’ that has been seen, then for each word we check if it is ‘crypto’, and if so increment the counter
  - The Parallel solution would be to process both documents at the same time, then add the counters together as a final step

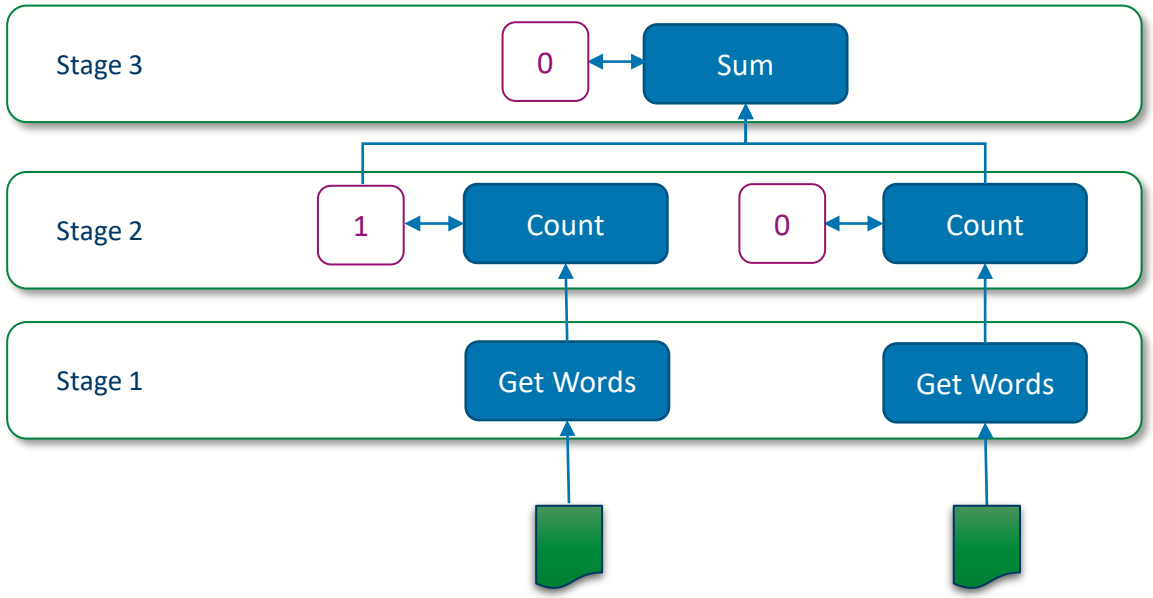




Additional Stages

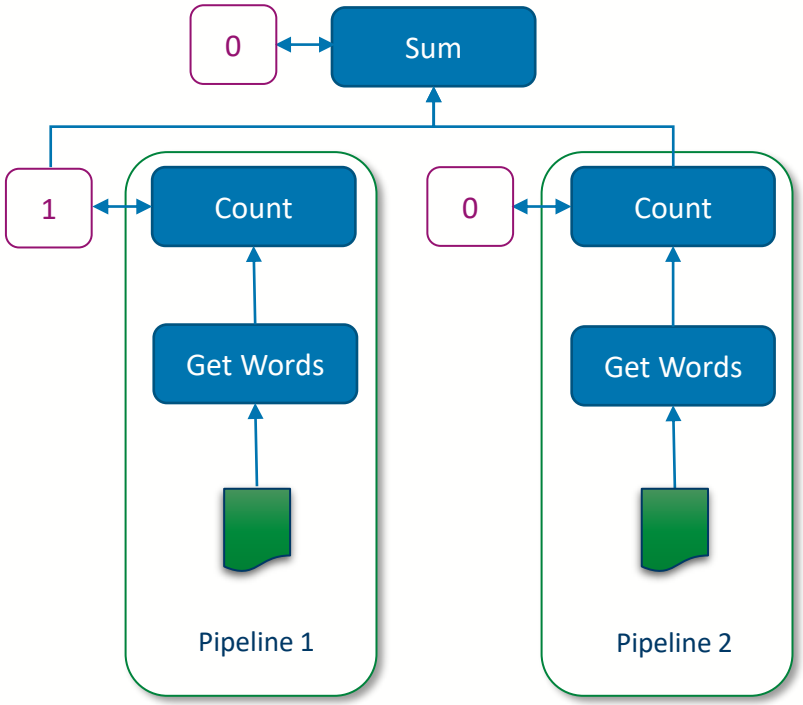
- As we convert to parallel solutions, we need to account for the fact that computation is happening locally in multiple places, requiring us to perform some sort of merge or aggregation of those intermediate outputs

- This usually takes the form of an additional stage of the processing that performs this aggregation (stage 3 in this case)
  - Importantly, this additional stage adds cost to the whole system, and often cannot be performed until the previous stage has completed



Data Splitting and Load Balancing

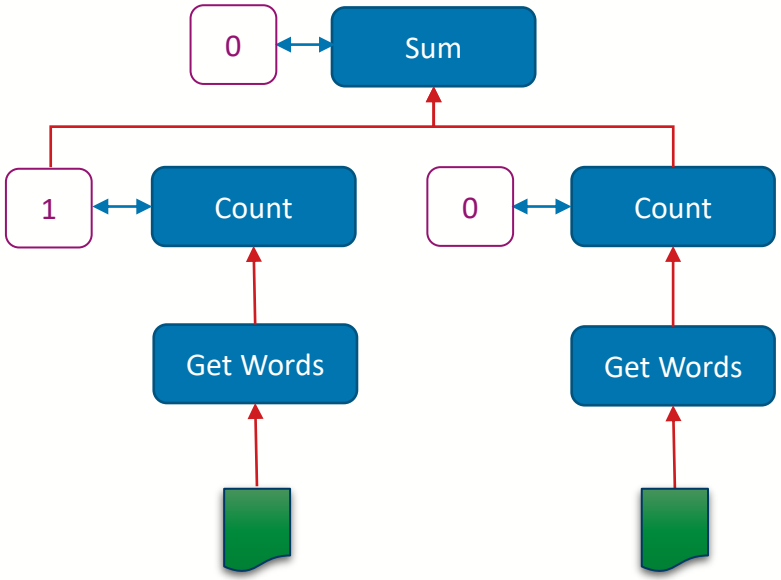
- To enable parallelism, we need to split our data into n chunks, where n is greater than the number of parallel ‘pipelines’ we have
  - In this case it was easy, as our items were already split into documents
- To make effective use of our resources, we want each pipeline to have the same amount of work to do, since if one pipeline finishes early then it will sit idle until the other is done
  - Note that in this example the document processed by pipeline 2 was shorter, and so it finished before pipeline 1



Communication and Intermediate Storage Overheads

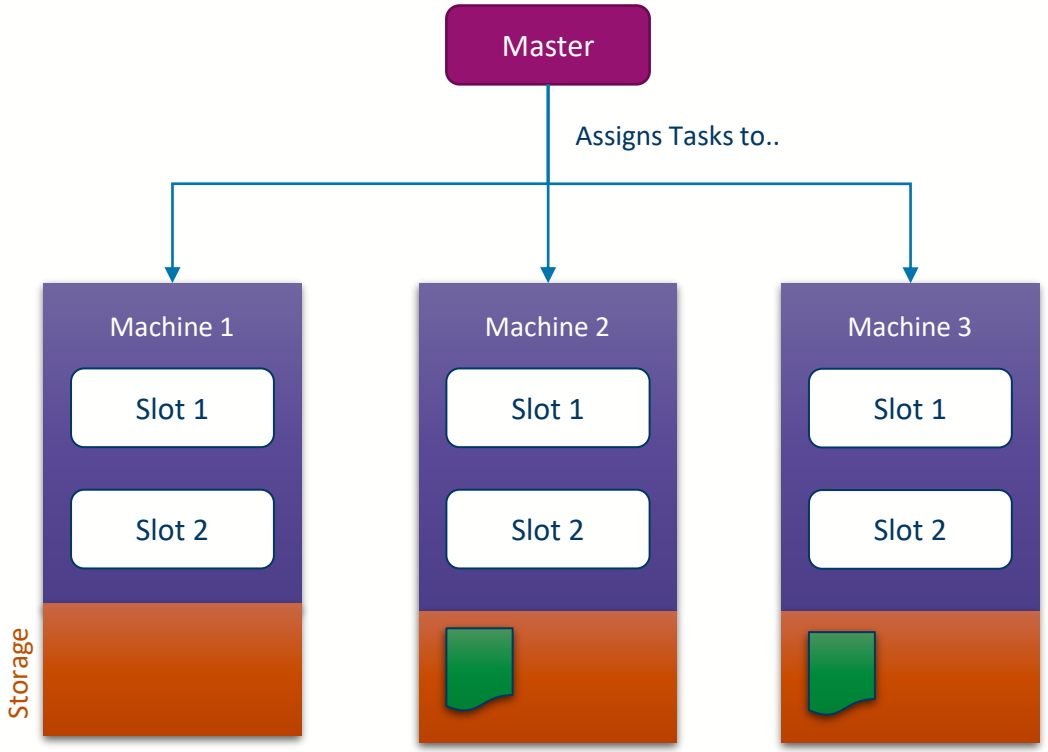
- As we add more tasks and stages to our system, we have more communication links between those tasks

- This may or may not be a problem depending on how data transfer between tasks is handled
  - If both tasks are hosted on the same physical machine the cost of this will depend on storage speed (see the hardware primer)
  - If on different machines we then need to add network transfer time



Orchestration

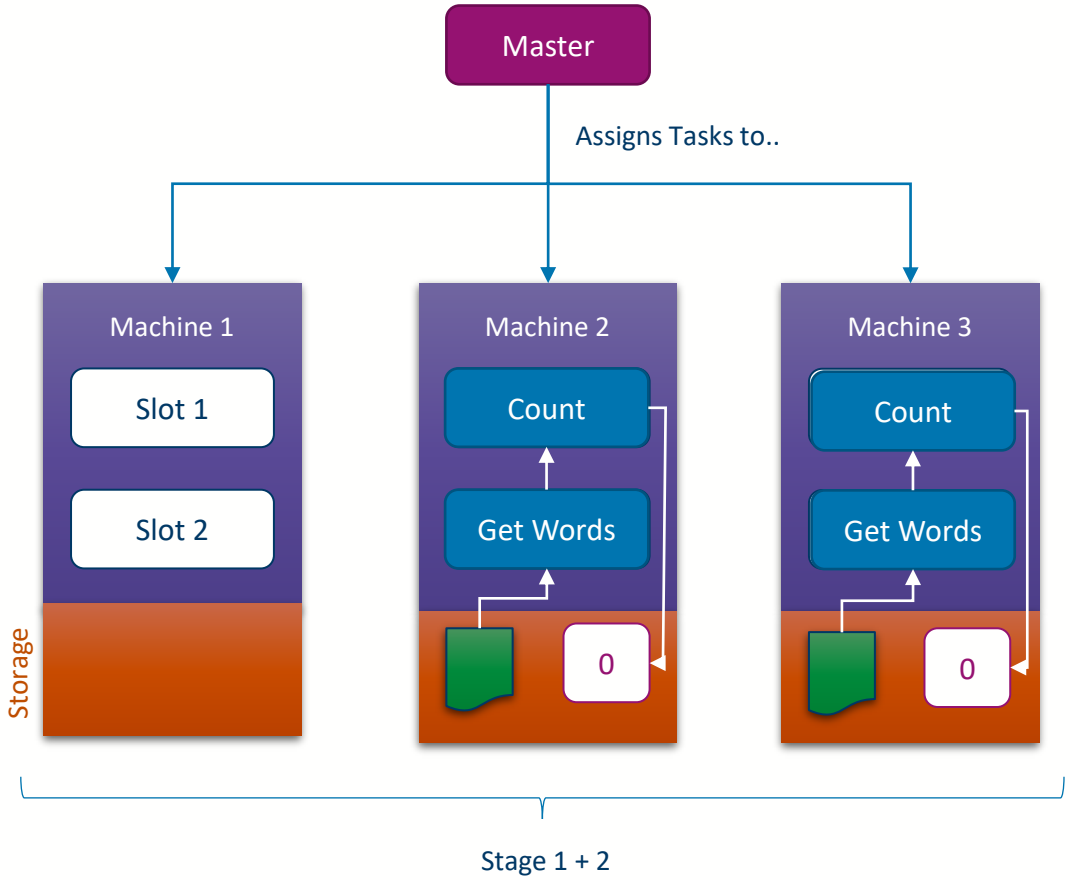
- As our architecture becomes more complex we need additional software to create the tasks
  - We usually call these master services
- A master service is responsible for allocating tasks to machines
  - Imagine a physical machine has a number of free slots on it, the master can plug tasks into those slots



University of Glasgow

Orchestration

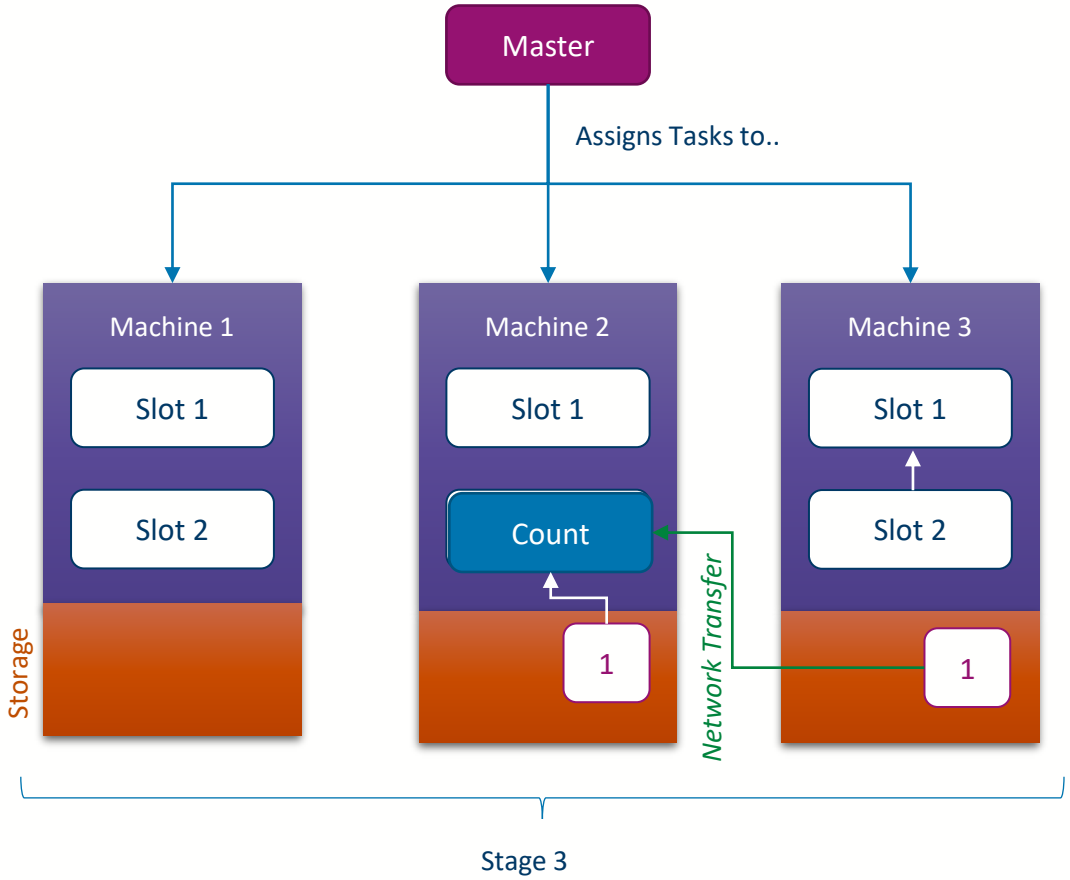
- As our architecture becomes more complex we need additional software to create the tasks
  - We usually call these master services
- A master service is responsible for allocating tasks to machines
  - Imagine a physical machine has a number of free slots on it, the master can plug tasks into those slots



University of Glasgow

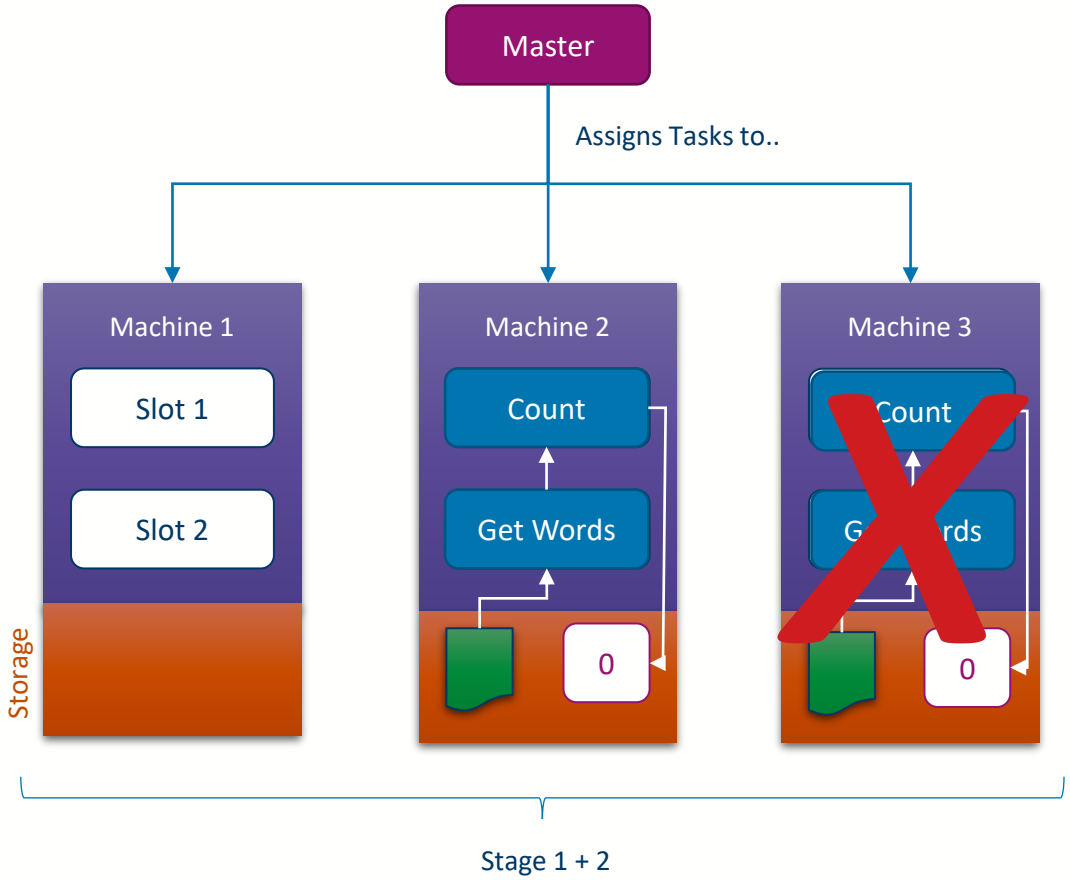
Orchestration

- As our architecture becomes more complex we need additional software to create the tasks
  - We usually call these master services
- A master service is responsible for allocating tasks to machines
  - Imagine a physical machine has a number of free slots on it, the master can plug tasks into those slots



Fault Tolerance

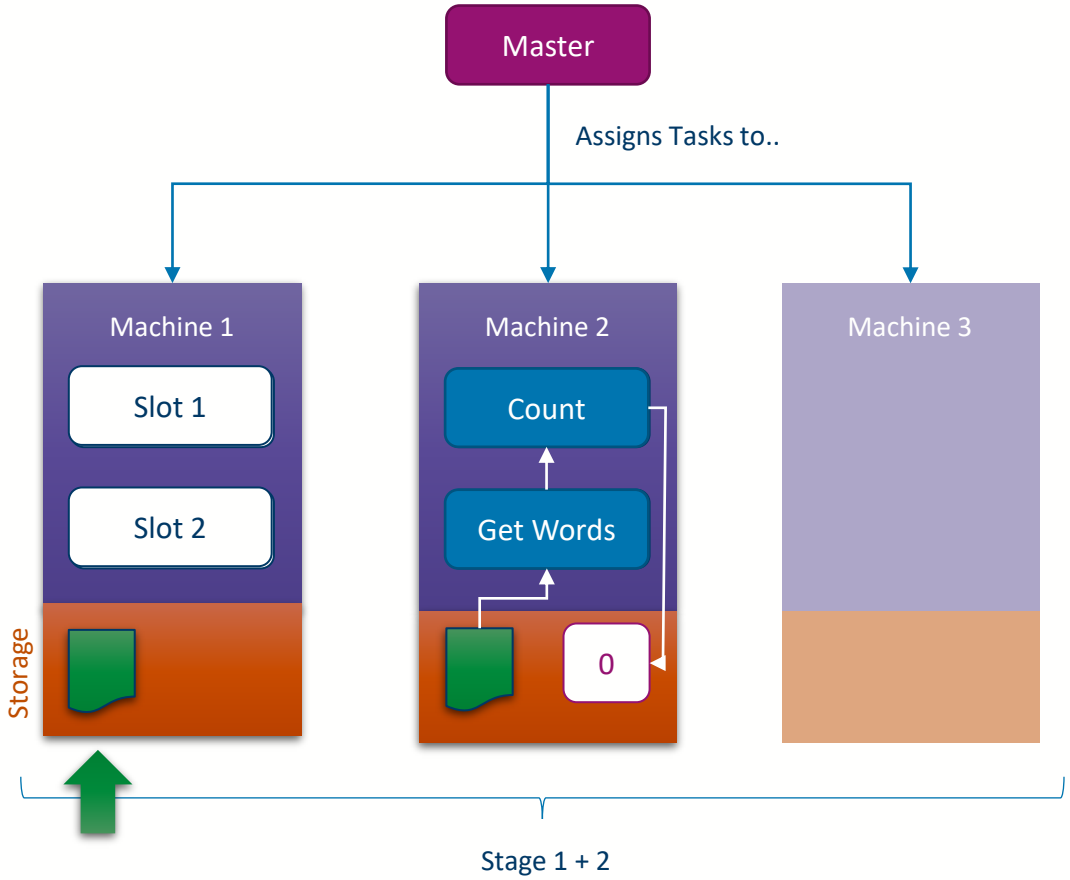
- What happens if a machine fails mid processing?
- We should ideally re-allocate the work that node was doing to a new machine
  - Need to get a fresh copy of the data to be processed
  - … then restart the tasks
- This can be orchestrated by the master service



University of Glasgow

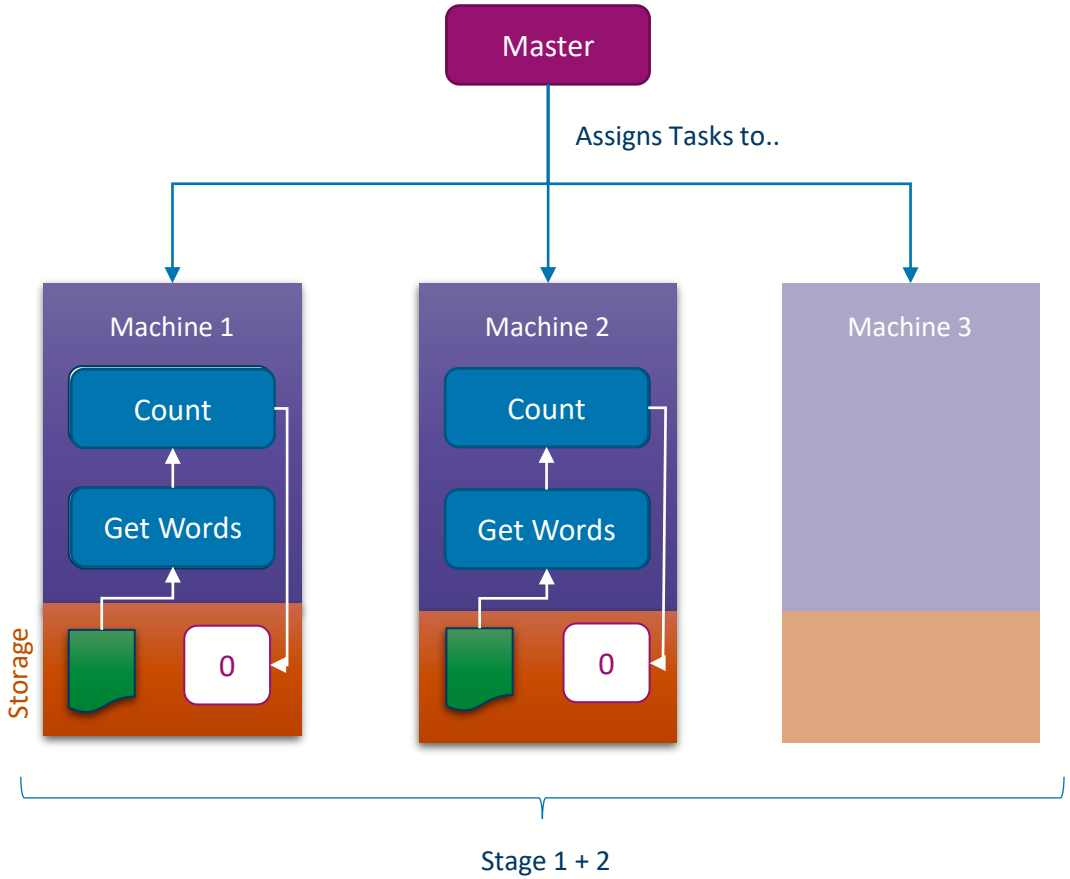
Fault Tolerance

- What happens if a machine fails mid processing?
- We should ideally re-allocate the work that node was doing to a new machine
  - Need to get a fresh copy of the data to be processed
  - … then restart the tasks
- This can be orchestrated by the master service



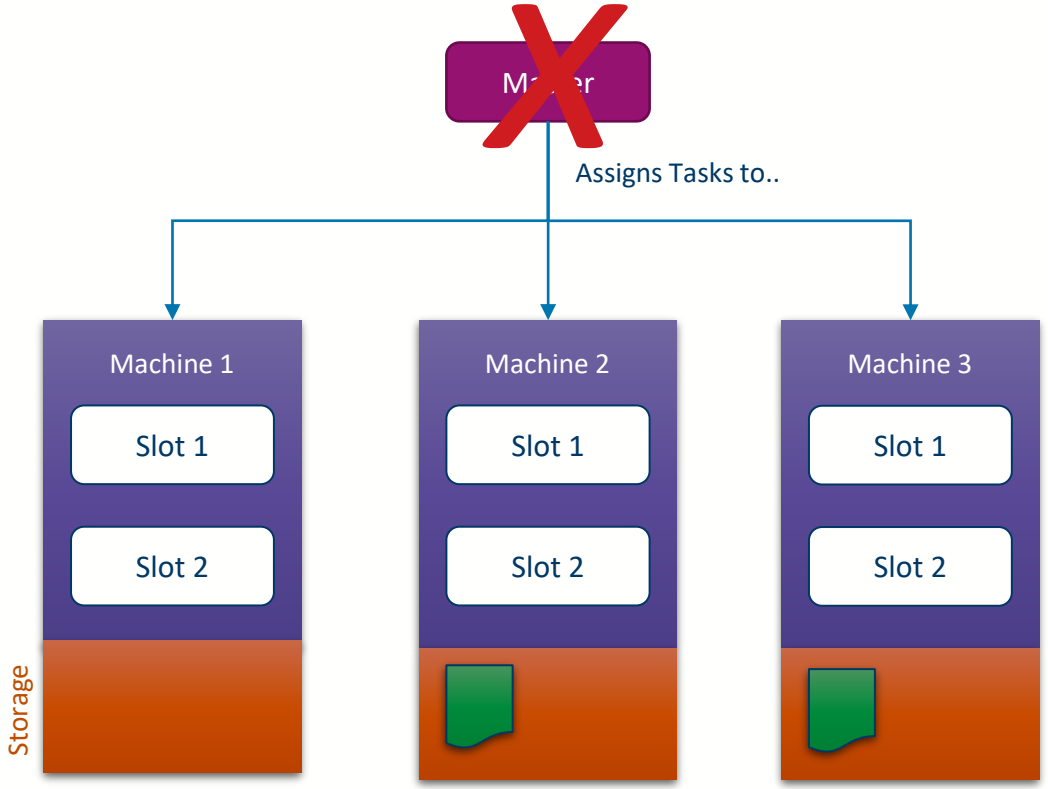
Fault Tolerance

- What happens if a machine fails mid processing?
- We should ideally re-allocate the work that node was doing to a new machine
  - Need to get a fresh copy of the data to be processed
  - … then restart the tasks
- This can be orchestrated by the master service



Master Failure

- However, the master service is also running on a machine, so what happens if the master dies?
  - … we likely cannot easily recover
- Unless we have multiple masters
  - But now we need to worry about conflict resolution amongst masters
  - Also ‘wasting’ more resources on management

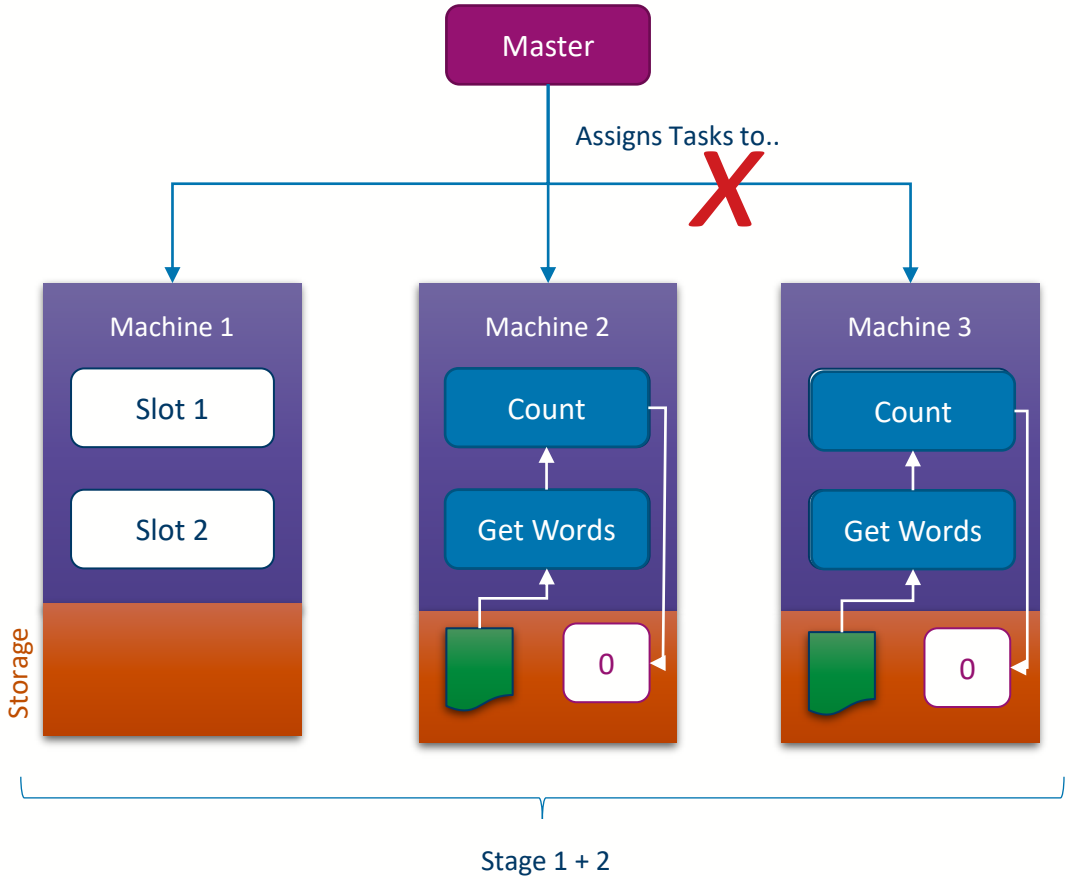


University of Glasgow

Network Failure

- What happens if the master cannot communicate with a machine?
  - Is the machine ok or dead?
  - Maybe its only a temporary glitch?

- If a machine is actually offline we need to re-allocate its work, but if its just out of contact we might end up doing the same work multiple times



University of Glasgow

Complexity, Complexity, Complexity

- In summary, moving to a parallel processing environment adds many new challenges that we need to solve
  - Load Balancing
  - Overheads
    - New Stages
    - Transfer Latency
  - Orchestration
    - Task Assignment
    - Machine Failures
    - Master Failures
    - Network Failures
  - … and more!

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers
@UofGlasgow